



青岛科技大学
Qingdao University of Science & Technology

数据结构与算法分析2

第8章 贪心法2—算法案例

梁宏涛

lht@qust.edu.cn

周二&周四上午

1

第8章—贪心法

提纲 CONTENTS

8.1 贪心法概述

8.2 区间问题

8.5 零钱兑换问题

8.3 背包问题

8.6 哈夫曼编码

8.4 田忌赛马问题

8.7 拟阵*

第8章—贪心法

提纲 CONTENTS

8.1 贪心法概述

8.2 区间问题

8.5 零钱兑换问题

8.3 背包问题

8.6 哈夫曼编码

8.4 田忌赛马问题

8.7 拟阵*



8.4 田忌赛马问题

问题描述 (POJ2287)：二千多年前的战国时期，齐威王与大将田忌赛马。双方约定每人各出**300**匹马，并且在上、中、下三个等级中各选一匹进行比赛，由于齐威王每个等级的马都比田忌的马略强，比赛的结果可想而知。现在双方各 **n** 匹马，依次派出一匹马进行比赛，每一轮获胜的一方将从输的一方得到**200**银币，平局则不用出钱，田忌已知所有马的速度值并可以安排出场顺序，问他如何安排比赛获得的银币最多。

输入格式：输入包含多个测试用例，每个测试用例的第一行正整数 **n** ($n \leq 1000$) 马的数量，后两行分别是 **n** 个整数，表示田忌和齐威王的马的速度值。输入 **$n=0$** 结束。

输出格式：每个测试用例输出一行，表示田忌获得的最多银币数。

输入样例:

3

92 83 71

95 87 74

2

20 20

20 20

2

20 19

22 18

0

样例输出:

200

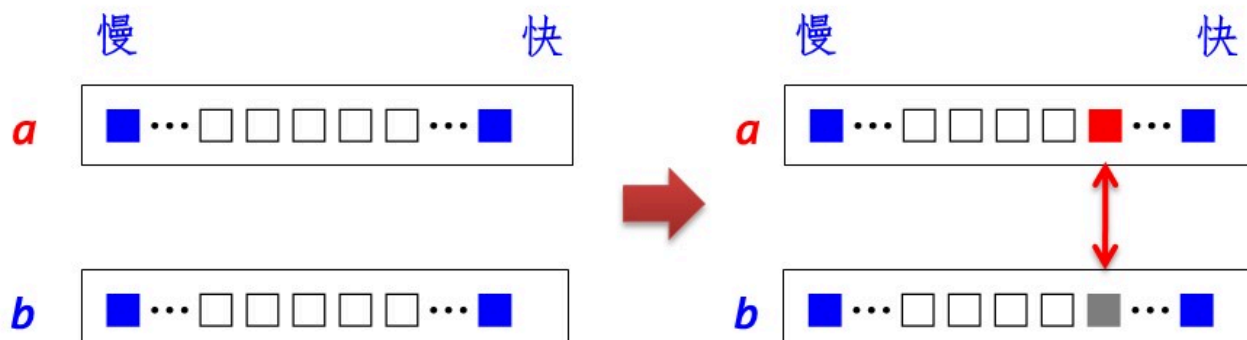
0

0

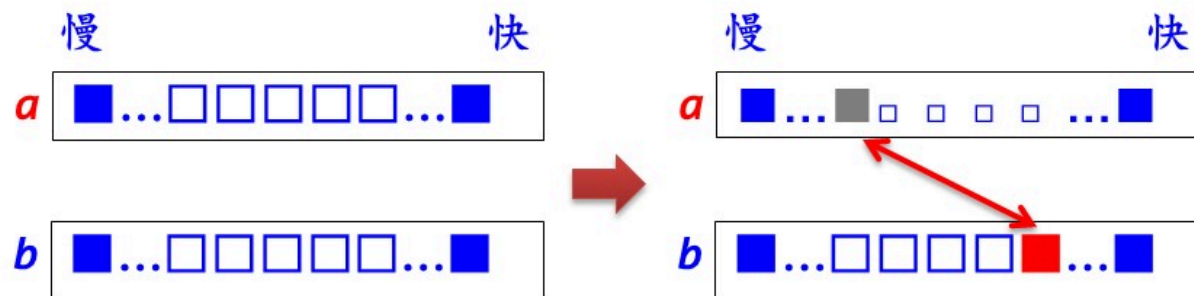
解

田忌的马速度用数组 a 表示，齐威王的马速度用数组 b 表示，将 a 、 b 数组递增排序。采用常识性的贪心思路，分以下几种情况：

(1) 田忌最快马比齐威王最快马快即 $a[\text{righta}] > b[\text{rightb}]$ ，则两者比赛（两个最快的马比赛），田忌赢。因为此时田忌最快的马一定赢，而选择与齐威王最快的马比赛对于田忌来说是最优的。



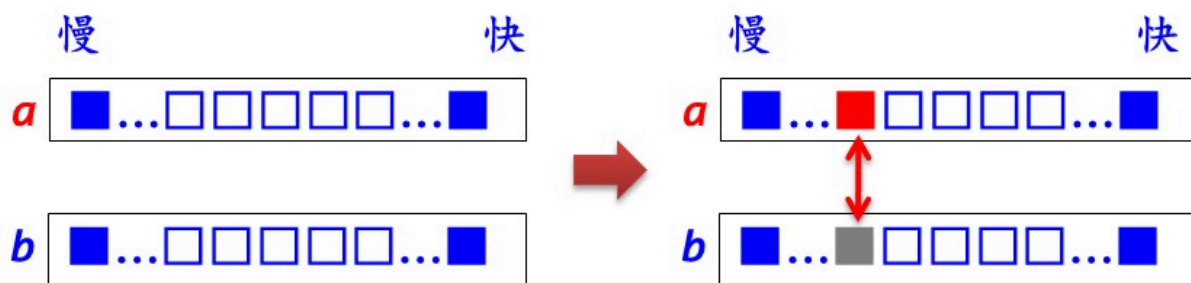
(2) 田忌最快马比齐威王最快马慢即 $a[righa] < b[righb]$, 则选择田忌最慢的马与齐威王最快的马比赛, 田忌输。因为齐威王最快的马一定赢, 而选择与田忌最慢的马比赛对于田忌来说是最优的。



$a[righa] < b[righb]: ans -= 200$

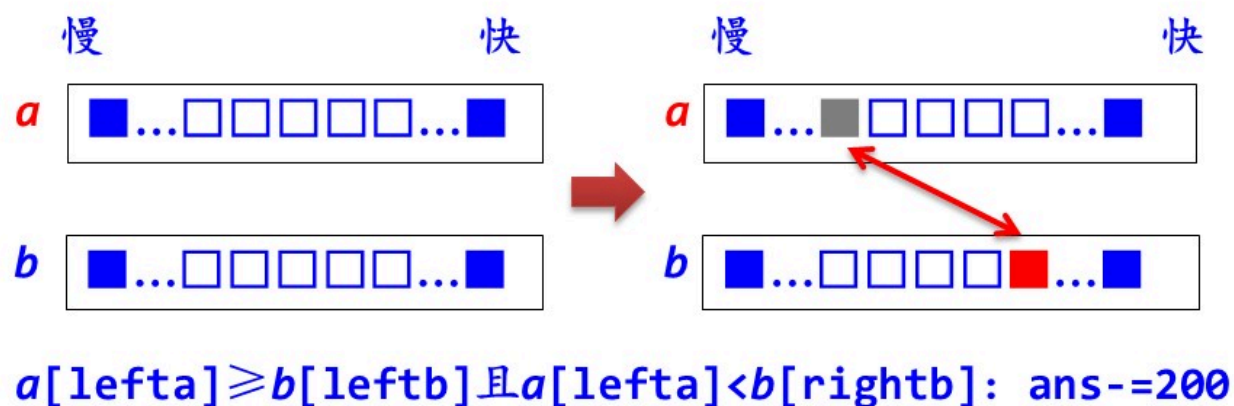
(3) 田忌最快马与齐威王最快马速度相同 $a[\text{righta}] = b[\text{rightb}]$, 又分为以下3种情况:

① 田忌最慢的马比齐威王最慢的马快即 $a[\text{lefta}] > b[\text{leftb}]$, 则两者比赛（两个最慢的马比赛），田忌赢。因为此时齐威王最慢的马一定输，而选择与田忌最慢的马比赛对于田忌来说是最优的。



$a[\text{lefta}] > b[\text{leftb}]: \text{ans} += 200$

② 田忌最慢的马比齐威王最慢的马慢，并且田忌最慢的马比齐威王最快的马慢，即 $a[\text{left}a] \leq b[\text{left}b]$ 且 $a[\text{left}a] < b[\text{right}b]$ ， 则选择田忌最慢的马与齐威王最快的马比赛，田忌输。因为此时田忌最慢的马一定输，而选择与齐威王最快的马比赛对于田忌来说是最优的。



③ 其他情况: $a[\text{righta}] = b[\text{rightb}]$ 且 $a[\text{lefta}] \leq b[\text{leftb}]$
且 $a[\text{lefta}] \geq b[\text{rightb}]$, 说明比赛区间的所以马的速度全部相同, 任何两匹马比赛都没有输赢。

每种情况对于田忌来说都是最优的, 因此最终获得的比赛方案也一定是最优的。

SD

```
1. #include<stdio.h>
2. #include<algorithm>
3. using namespace std;
4. #define MAX 1001
5. int n;
6. int a[MAX];
7. int b[MAX];
8. int ans;           //田忌获得的最多银币数
```

```
1. void solve() { //求解算法
2.     sort(a,a+n); //对a递增排序
3.     sort(b,b+n); //对b递增排序
4.     ans=0;
5.     int lefta=0,leftb=0;
6.     int righta=n-1,rightb=n-1;
7.     while (lefta<=righta) { //比赛直到结束
8.         if (a[righta]>b[rightb]) {
9.             //田忌最快的马比齐威王最快的马快，两者比赛
10.            ans+=200;
11.            righta--;
12.            rightb--;
13.        }
```



```
1. else if (a[righta]<b[rightb]) {  
2.           //田忌最快的马比齐威王最快的马慢  
3.   ans-=200; //选择田忌最慢的马与比齐威王最快的马比赛  
4.   lefta++;  
5.   rightb--;  
6. }
```

```
1.     else {           //田忌最快的马与齐威王最快的马的速度相同
2.         if (a[lefta]>b[leftb]) {
3.             //田忌最慢的马比齐威王最慢的马快，两者比赛
4.             ans+=200;
5.             lefta++;
6.             leftb++;
7.         }
8.         else {
9.             if (a[lefta]<b[rightb])
10.                //否则，用田忌最慢的马与齐威王最快的马比赛
11.                ans-=200;

12.            lefta++;
13.            rightb--;
14.        }
15.    }
16. }
17. }
```

```
1. int main() {  
2.     while (true) {  
3.         scanf("%d",&n);  
4.         if (n==0) break;  
5.         for (int i=0;i<n;i++)  
6.             scanf("%d",&a[i]);  
7.         for (int j=0;j<n;j++)  
8.             scanf("%d",&b[j]);  
9.         solve();  
10.        printf("%d\n",ans);  
11.    }  
12.    return 0;  
13. }
```

 执行用时为**32ms**，内存消耗为**104KB**。

8.5 零钱兑换问题

 **问题描述：**有面额分别是 $c^0, c^1, \dots, c^k$ ($c \geq 2, k$ 为非负整数) 的 $k+1$ 种硬币，每种硬币个数可以看成无限多，求兑换 A 金额的最少硬币个数。

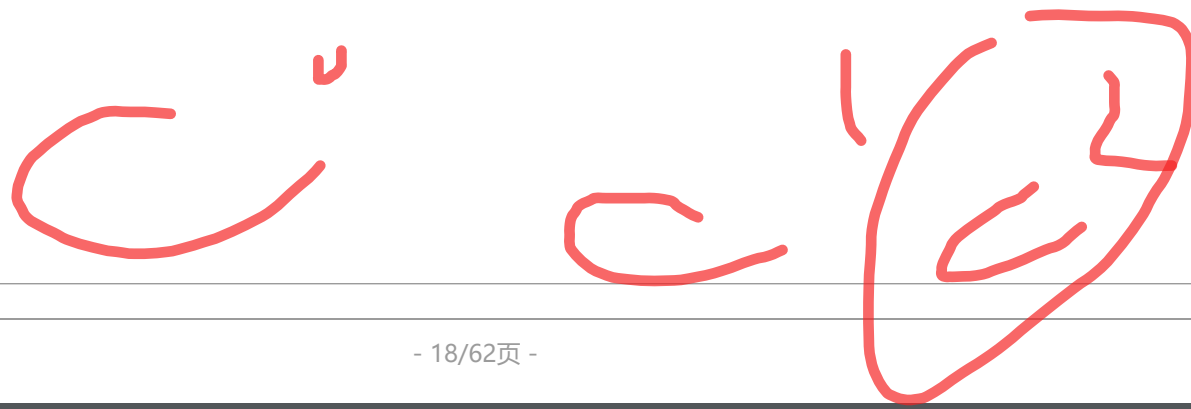
~~找大钱~~

解

采用贪心法，贪心策略是尽可能选择面额大的硬币进行兑换，例如， $c=2$ ， $k=3$ ，面额分别是1、2、4和8， $A=23$ 的兑换过程如下：

- ① 选择面额为8的硬币，兑换的硬币个数= $A/8=2$ ，
剩余金额= $23-2\times 8=7$ 。
- ② 选择面额为4的硬币，兑换的硬币个数= $A/4=1$ ，
剩余金额= $7-4\times 1=3$ 。
- ③ 选择面额为2的硬币，兑换的硬币个数= $A/2=1$ ，
剩余金额= $3-2\times 1=1$ 。
- ④ 选择面额为1的硬币，兑换的硬币个数= $A/1=1$ ，
剩余金额= $1-1\times 1=0$ 。

```
int greedy(int c,int k,int A) { //求解算法
    int ans=0;
    int curm=(int)Math.pow(c,k); //初始取最大面额的硬币
    while(A>0) {
        int curs=A/curm; //求面额为curm的硬币个数curs
        ans+=curs; //累计硬币个数
        A-=curs*curm; //剩余金额为A
        curm/=c; //试探其他硬币
    }
    return ans;
}
```





- 并非任何面额的零钱兑换问题都可以用上述贪心法求最少的硬币个数！
- 例如，硬币面额分别为**1**、**4**、**5**和**10**时， **$A=18$** ，采用贪心法求出的兑换硬币个数为**5**（**1**个面额为**10**、**1**个面额**5**和**3**个面额为**1**的硬币），而实际最优解是**3**（**1**个面额为**10**和**2**个面额**4**的硬币）。

8.6 哈夫曼编码

 **问题描述：** 设需要编码的字符集为 $\{d_0, d_1, \dots, d_{n-1}\}$ ，它们出现的频率为 $\{w_0, w_1, \dots, w_{n-1}\}$ ，应用哈夫曼树构造最优的不等长的由0、1构成的编码方案（哈夫曼编码）。

解

哈夫曼树 $\Rightarrow$ 哈夫曼编码

贪心策略：频率越低编码越长，频率越高编码越短。
合并次数越多编码越长。

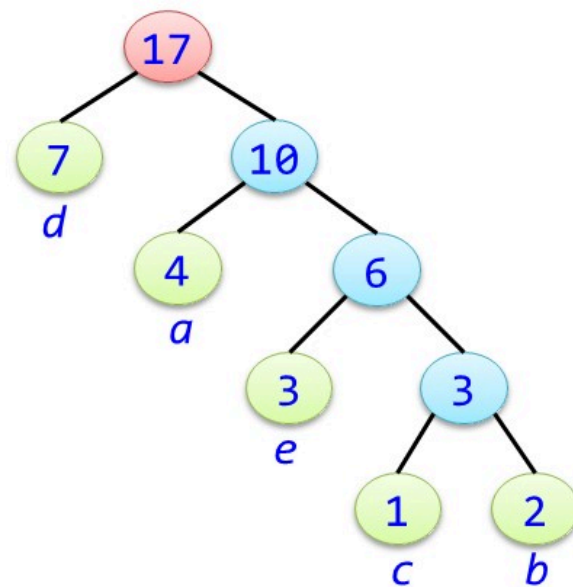
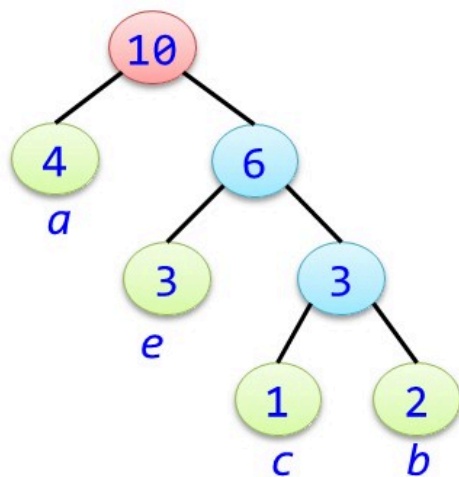
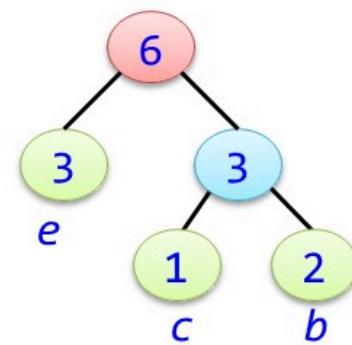
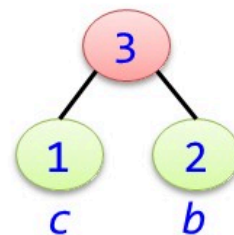
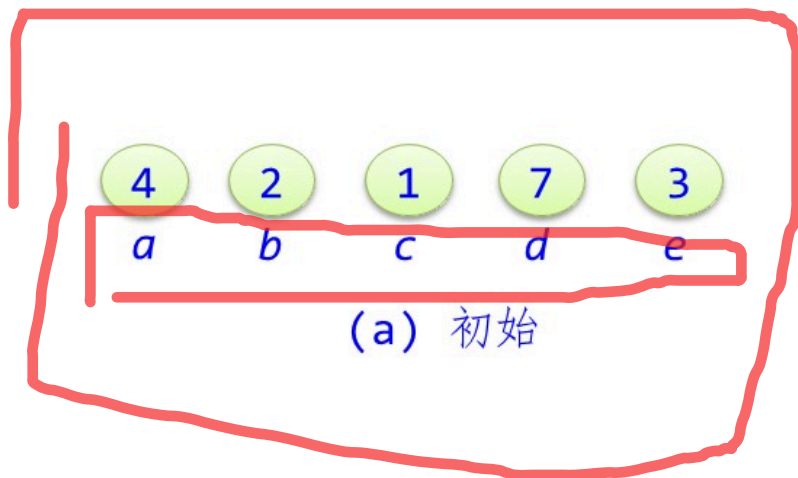
证明略

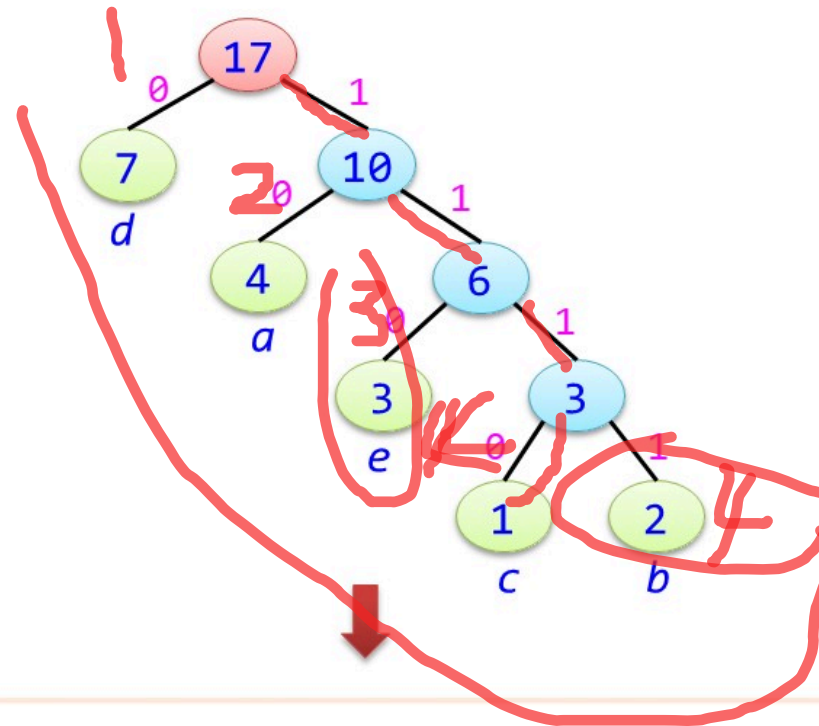
构造一棵哈夫曼树

- ① 由 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构造 n 棵只有一个叶子结点的二叉树，得到一个二叉树集合 $F=\{T_1, T_2, \dots, T_n\}$ 。
- ② 在 F 中选取根结点的权值最小和次小的两棵二叉树作为左、右子树构造一棵新的二叉树，这棵新的二叉树根结点的权值为其左、右子树根结点权值之和。即合并两棵二叉树为一棵二叉树。
- ③ 重复步骤②，当 F 中只剩下一棵二叉树时，这棵二叉树便是所要建立的哈夫曼树。

构造哈夫曼编码

- 利用哈夫曼树构造的用于通信的二进制编码称为哈夫曼编码。
- 哈夫曼树中从根到每个叶子都有一条路径，对路径上的各分支约定指向左子树根的分支表示“0”码，指向右子树的分支表示“1”码，取每条路径上的“0”或“1”的序列作为和各个叶子对应的字符的哈夫曼编码。





a: 10

b: 1111

c: 1110

d: 0

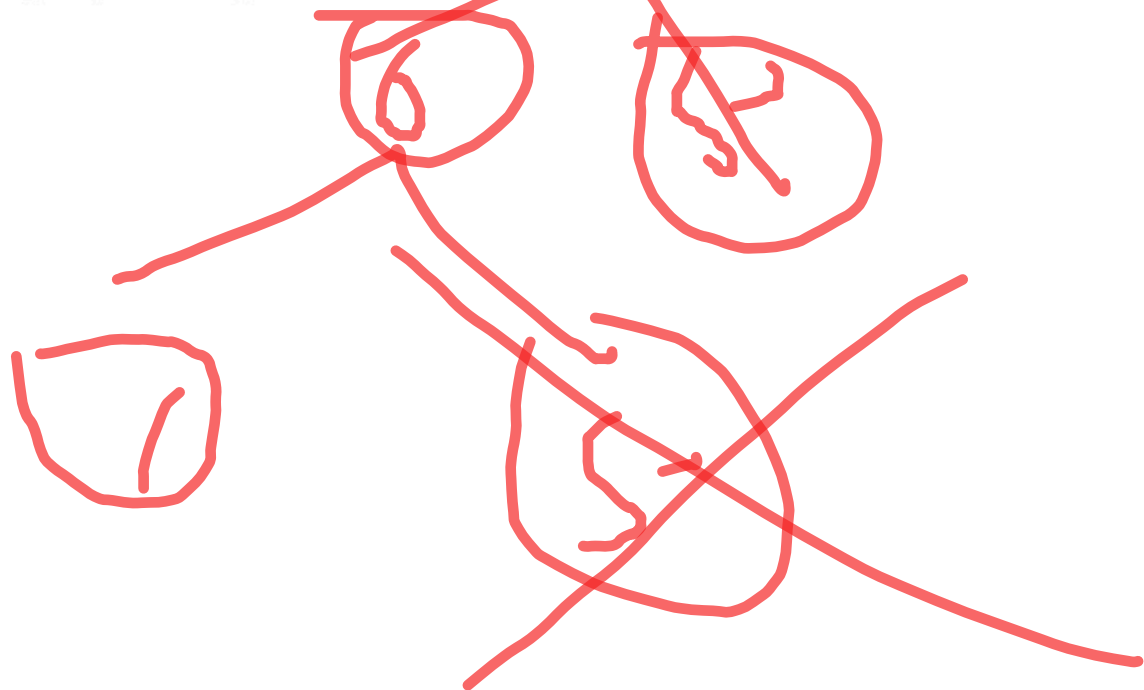
e: 110

$$WPL = (1+2) \times 4 + 3 \times 3 + 4 \times 2 + 7 \times 1 = 36$$

哈夫曼编码

主观题 10分

“青岛科技大学” 6个汉字的频率为{5,73,9,4,1};
请绘制哈夫曼树，并对汉字进行编码。





【例8.6】最后一块石头的重量

(LeetCode1046★)。有 n ($1 \leq n \leq 30$) 块石头，每块石头的重量都是正整数（重量为 $1 \sim 1000$ ）。每一回合从中选出两块最重的石头，然后将它们一起粉碎。假设石头的重量分别为 x 和 y ，且 $x \leq y$ 。那么粉碎的可能结果如下：

- ① 如果 $x=y$ ，那么两块石头都会被完全粉碎。
- ② 如果 $x \neq y$ ，那么重量为 x 的石头将会完全粉碎，而重量为 y 的石头新重量为 $y-x$ 。

最后最多只会剩下一块石头，求此石头的重量，如果没有石头剩下结果为0。

要求设计如下方法：

```
int lastStoneWeight(vector<int>& stones) {}
```

解

- 选石头的过程与构造哈夫曼树的过程类似，只是这里选的是两块最重的石头。
- 用优先队列（大根堆）求解，每次出队两块最重的石头 x 和 y ，然后将 $x-y$ 进队，直到仅有一块石头为止。

```

1. class Solution {
2. public:
3.     int lastStoneWeight(vector<int>& stones) {
4.         int n=stones.size();
5.         if(n==1) return stones[0];
6.         priority_queue<int> maxpq;           //定义大根堆
7.         for(int i=0;i<n;i++)
8.             maxpq.push(stones[i]);          //所有石头进队
9.         int x=0,y=0;
10.        while(!maxpq.empty()) {
11.            x=maxpq.top(); maxpq.pop();
12.            if(maxpq.empty())                 //若x是最后的石头
13.                return x;
14.            y=maxpq.top(); maxpq.pop();       //若x不是最后石头
15.            maxpq.push(x-y);                  //新重量为 x-y( $x \geq y$ )
16.        }
17.        return x;
18.    }
19.

```

EXE

执行用时为**4ms**，内存消耗为**7.4MB**。

28

8.7 拟阵*

8.7.1 拟阵概述

拟阵 M 是一个满足如下条件的有序对 $M=(S, I)$:

(1) S 是一个有限集合。

(2) I 是 S 的一个具有遗传性质的非空子集族（集合的集合）。遗传性质是指如果 $B \in I$ 且 $A \subseteq B$, 则 $A \in I$, 即若 $B \in I$, 则 B 是 S 的独立子集, 且 B 是任意子集也是 S 的独立子集。空集必是 I 的成员。

(3) I 具有交换性质。交换性质是指如果 $A \in I$, $B \in I$ 且 $|A| < |B|$, 则存在某个元素 $x \in B - A$, 使得 $A \cup \{x\} \in I$ 。

其中(2)提示了已知集合 B 找到其子集的办法, (3)提示了已知集合 B 构造集合 A 的办法。

【例8.7】 对于非空无向图 $G=(V, E)$ ，定义 $M[G]=(S[G], I[G])$ ，其中 $S[G]$ 为 E 即 G 的边集，如果 $A \subseteq E$ 且 A 是无环的，则 $A \in I[G]$ ，也就是说一组边 A 是独立的当且仅当子图 $G[A]=(V, A)$ 构成一个森林。

证明 $M[G]$ 是一个拟阵。

非空无向图 $G=(V, E)$, $M[G]=(S[G], I[G])$, 其中 $S[G]$ 为 E 即 G 的边集, 如果 $A \subseteq E$ 且 A 是无环的, 则 $A \in I[G]$, 也就是说一组边 A 是独立的当且仅当子图 $G[A]=(V, A)$ 构成一个森林。证明 $M[G]$ 是一个拟阵。

证明

(1) 显然 $S[G]=E$ 是一个有限集合。

(2) $I[G]$ 具有遗传性质。对于任意 $B \in I[G]$, $A \subseteq B$, 则 $A \in I[G]$, 因为从一组无环的边中删除一些边不会出现新的环。

(3) $I[G]$ 具有交换性质。有 k 条边的森林恰好包含 $|V| - k$ 棵树，若 $G[A] = (V, A)$ 和 $G[B] = (V, B)$ 是 G 的森林，且 $|B| > |A|$ ，则 $G[A]$ 中有含 $|V| - |A|$ 棵树， $G[B]$ 中有含 $|V| - |B|$ 棵树。

由于 $G[B]$ 中的树比 $G[A]$ 中的树要少，必有边 $e \in G[B]$ 而 $e \notin G[A]$ ，而且连接 $G[A]$ 中的两棵树，将 e 添加到 A 中不会产生环，即 $A \cup \{e\} \in I[G]$ 。

定理8.3 一个拟阵 $M=(S, I)$ 中所有最大独立子集的大小都是相同的。

对于拟阵 $M=(S, I)$ ， S 中的每个元素 x 都有一个权值 $w(x)$ （权函数为 w ），一个独立子集 A 的权是它的所有元素的权之和，即，则称拟阵 M 为**加权拟阵**。

8.7.2 求加权拟阵最优子集的贪心算法

求加权（权值为 w ）拟阵 $M=(S, I)$ 的最优子集 A 的贪心算法框架

```
Set greedy(M,w) {                                     //M=(S, I)
    A=∅;                                                //∅表示空集{}
    将S中元素依权值w（大者优先）组成优先队列；
    while (S!=∅) {
        S.removeMax(x);                                //出队x
        if (A∪{x}∈I) A=A∪{x};
    }
    return A
}
```


引理8.4（拟阵具有贪心选择性质）。假设 $M=(S, I)$ 是一个具有权函数 w 的加权拟阵，且 S 被按权值的单调减顺序排列。设 x 是 S 的第一个使 $\{x\}$ 独立的元素。如果 x 存在，则存在 S 的一个包含 x 的最优子集 A 。

引理8.6（拟阵具有最优子结构性质）。设 x 为 S 中被作用于加权拟阵 $M=(S, I)$ 的greedy算法第一个选择的元素。找一个包含 x 的具有最大权值的独立子集的问题，可以归结为找出加权矩阵 $M'=(S', I')$ 的一个具有最大权值的独立子集问题，此处

$$S' = \{y \in S \mid \{x, y\} \in I\}, \quad I' = \{B \subseteq S - \{x\} \mid B \cup \{x\} \in I\}$$

其中， M' 权函数为（受限于 S' 的） M 的权函数（称 M' 为 M 的由 x 引起的收缩）。

8.7.3 带期限和惩罚的任务调度问题

讨论单处理器上具有期限和惩罚的**单位时间任务调度问题**，所谓单位时间任务是一个作业（如在计算机上执行的程序）恰好需要一个单位的时间来执行，为了简单，时间用**0**开始的整数表示。该问题的描述如下：

- (1) n 个单位时间任务的集合 $S = \{a_1, a_2, \dots, a_n\}$ 。
 - (2) n 个整数值的期限集合 $D = \{d_1, d_2, \dots, d_n\}$ ，即任务 a_i ($1 \leq i \leq n$) 要求在 d_i 之前完成。
 - (3) n 个非负的权值（惩罚代价）集合 $W = \{w_1, w_2, \dots, w_n\}$ 。如果 a_i 在 d_i 之前没有完成，就会有惩罚 w_i ；否则没有惩罚。
- 目标是找出一个 S 的调度，使得**总的惩罚最小**。

解

- 考虑一个给定的调度（即 n 个任务的一种执行顺序），该调度中若一个任务在限期之后完成的称其为迟任务，一个任务在限期之前完成称其为早任务。
- 一个任意的任务调度总可以安排成早任务优先的形式，其中早任务总是在迟任务之前完成，如果某个早任务 a_i 在迟任务之后 a_j ，那么交换 a_i 和 a_j 的次序，并不会影响 a_i 是早任务和 a_j 是迟任务。

- **最优调度方案：**早任务先于迟任务，并且按照期限的单调递增顺序对早任务进行调度。
- 寻找最优调度问题变成了一个寻找一个由最优调度中的早任务构成的集合**A**的问题，一旦**A**被确定之后，就可以按照期限的单调递增顺序列出**A**中的所有元素，从而得到实际的调度。

定理8.2 如果 S 是一个带期限的单位时间任务的集合, 并且 I 是所有独立的任务集构成的集合, 则对应的系统 (S, I) 是一个拟阵。



证明了单位时间任务调度问题采用上述贪心法是正确的!

【例8.9】如表8.4所示的7个作业，假设每个作业需要一个时间单位加工，采用贪心法求最小惩罚数。

作业编号 a_i	截止时间 d_i	惩罚值 w_i
1	4	70
2	2	60
3	4	50
4	3	40
5	1	30
6	4	20
7	6	10

解法1

- 采用基于拟阵的贪心算法框架 **greedy**(M, w) 求解，对应的加权拟阵 $M=(S, I)$ ，其中 S 是所有作业的集合， I 是早作业集合的集合，权值 w 为作业的惩罚值，该问题是 **最小化迟作业的惩罚之和** 问题（或者 **最大化早作业的奖励之和** 问题）。
- 贪心策略：**优先完成惩罚值大的作业。**
- 用 **ans** 表示最小惩罚数（初始为 **0**），按惩罚值 w_i 递减排序，排序结果为 **[4, 70], [2, 60], [4, 50], [3, 40], [1, 30], [4, 20], [6, 10]**。

■ 排序结果为 $[4, 70]$, $[2, 60]$, $[4, 50]$, $[3, 40]$, $[1, 30]$, $[4, 20]$, $[6, 10]$ 。

(1) $i=0$, 任务 a_1 为 $[4, 70]$, 其截止时间为 4, 选择第 4 天完成, a_1 添加到 A 中。

(2) $i=1$, 任务 a_2 为 $[2, 60]$, 其截止时间为 2, 选择第 2 天完成, a_2 添加到 A 中。

(3) $i=2$, 任务 a_3 为 $[4, 50]$, 其截止时间为 4, 选择第 3 天完成, a_3 添加到 A 中。

(4) $i=3$, 任务 a_4 为 $[3, 40]$, 其截止时间为 3, 选择第 1 天完成, a_4 添加到 A 中。

■ 排序结果为 $[4, 70]$, $[2, 60]$, $[4, 50]$, $[3, 40]$, $[1, 30]$, $[4, 20]$, $[6, 10]$ 。

(5) $i=4$, 任务 a_5 为 $[1, 30]$, 其截止时间为1, 第1天被占用, 不能完成该任务, 需要惩罚, $ans=30$ 。

(6) $i=5$, 任务 a_6 为 $[4, 20]$, 其截止时间为4, 第1~4天均被占用, 不能完成该任务, 需要惩罚, $ans=30+20=50$ 。

(7) $i=6$, 任务 a_7 为 $[6, 10]$, 其截止时间为6, 选择第6天完成, a_7 添加到A中。



最小惩罚数 $ans=50$, 对应的独立任务集 $A=\{a_1, a_2, a_3, a_4, a_7\}$, 最终的一个最优调度是 $\{a_1, a_2, a_3, a_4, a_7, a_5, a_6\}$ 。

解法2

- 也可以将这样的贪心解法，将全部作业按截止时间 d 递减排序，排序结果为 $[6, 10]$, $[4, 70]$, $[4, 50]$, $[4, 20]$, $[3, 40]$, $[2, 60]$, $[1, 30]$ 。
- 用 ans 表示最小惩罚数（初始为0），最大的截止时间为6， day 从6到1循环，每一天选择早任务（截止时间 $\geq day$ ）中惩罚值最大的任务来完成。

■ 按截止时间 d 递减排序: $[6, 10]$, $[4, 70]$, $[4, 50]$, $[4, 20]$, $[3, 40]$, $[2, 60]$, $[1, 30]$ 。

(1) $\text{day}=6$: 截止时间大于等于6的任务为 $[6, 10]$, 在第6天完成作业 $[6, 10]$ 。

(2) $\text{day}=5$: 截止时间大于等于5的任务为空。

(3) $\text{day}=4$: 截止时间大于等于4的任务为 $[4, 70]$, $[4, 50]$ 和 $[4, 20]$, 在第4天完成惩罚值最大的作业 $[4, 70]$ 。

(4) $\text{day}=3$: 截止时间大于等于3的任务为 $[4, 50]$, $[4, 20]$ 和 $[3, 40]$, 在第3天完成惩罚值最大的作业 $[4, 50]$ 。

■ 按截止时间 d 递减排序: $[6, 10]$, $[4, 70]$, $[4, 50]$, $[4, 20]$, $[3, 40]$, $[2, 60]$, $[1, 30]$ 。

(5) $\text{day}=2$: 截止时间大于等于2的任务为 $[4, 20]$, $[3, 40]$ 和 $[2, 60]$, 在第2天完成惩罚值最大的作业 $[2, 60]$ 。

(6) $\text{day}=1$: 截止时间大于等于1的任务为 $[4, 20]$, $[3, 40]$ 和 $[1, 30]$, 在第1天完成惩罚值最大的作业 $[3, 40]$ 。



不能完成的任务是 $[4, 20]$ 和 $[1, 30]$, 对应的最小惩罚数
 $\text{ans}=20+30=50$, 一个最优调度是 $\{a_4, a_2, a_3, a_1, a_7, a_5, a_6\}$ 。



【例8.10】超市问题（POJ1456，时间限制为2000ms，空间限制为65536KB）。一家超市有一组商品**Prod**在售。每件商品有一个截止时间**dx**和销售利润**px**，该商品在截止时间**dx**之前售出时会获得利润**px**。每件商品的销售时间恰好是一个单位。销售的商品**Sell**是**Prod** 的有序子集，其销售利润是 $\text{Profit}(\text{Sell}) = \sum_{x \in \text{Sell}} (px)$ 。最优销售调度是具有最大利润的调度。

例如, $\text{Prod}=\{a, b, c, d\}$

$(p_a, d_a)=(50, 2)$

$(p_b, d_b)=(10, 1)$

$(p_c, d_c)=(20, 2)$

$(p_d, d_d)=(30, 1)$ 。

所有可能的销售调度如表所示, 例如,

$\text{Sell}=\{d, a\}$ 表示商品 d 的销售从时间 0 开始到时间 1 结束(销售时间为 1), 商品 a 的销售从时间 1 开始到时间 2 结束(销售时间为 2), 每一个商品都在其截止时间前售出, 是最优销售调度, 其利润为 80 。请你编写一个程序, 从输入文件中读取各组商品, 并计算每组商品的最优销售利润。

销售调度	利润
{a}	50
{b}	10
{c}	20
{d}	30
{b, a}	60
{a, c}	70
{c, a}	70
{b, c}	30
{d, a}	80
{d, c}	50

输入格式：以整数 n ($0 \leq n \leq 10000$) 开始，表示商品数量，接下来是 n 个 $p_i \ d_i$ ($1 \leq p_i \leq 10000$ 和 $1 \leq d_i \leq 10000$) 整数对，每对表示第 i 个商品的利润和销售期限，中间可能有一个或者多个空格。输入数据以文件结尾结束并保证正确。

输出格式：对于每组商品，在一行中输出一个整数表示最优销售利润。

输入样例:

4 50 2 10 1 20 2 30 1

7 20 1 2 1 10 3 100 2 8 2

5 20 50 10

输出样例:

80

185

50

解法1

- 贪心策略：优先选择利润大的商品，在空时间中选择尽可能晚的时间。
- 用 `prod` 数组存放全部商品的数据，将 `prod` 按利润递减排序，用 `days[0..maxd]` 数组标记 `0~maxd` 时间的占用情况（销售商品的有效时间是 `1~maxd`），初始时全部元素置为 `false`。
- `i` 从 `0` 到 `n-1` 遍历 `prod`，对于商品 `i`，`j` 从其最后截止时间 `prod[i].d` 到 `0` 的顺序找 `days[j]` 为 `false` 的空时间，若找到了，则在时间 `j` 销售商品 `i`，将其利润累计到 `ans` 中，同时置 `days[j]` 为 `true`（表示时间 `j` 被占用）。最后返回 `ans` 即可。


```

#include<iostream>
#include<queue>
#include<vector>
#include<algorithm>
using namespace std;
#define MAXN 10005

struct Products {                                //商品的类型
    int d;                                       //截止时间
    int p;                                       //利润
    bool operator<(const Products&o) const {    //用于按p递减排序
        return p>o.p;
    }
};
Products prod[MAXN];                             //存放全部商品

```

```

int schedule(int n,int maxd) {           //贪心算法
    sort(prod,prod+n);                 //按p递减排序
    int ans=0;                          //答案
    vector<bool> days(maxd+1,false);    //标志时间是否已经占用
    for(int i=0;i<n;i++) {              //遍历商品
        int j=prod[i].d;
        for(;j>0;j--) {                 //查找截止时间之前的最晚空时间
            if(!days[j]) {             //找到空时间
                days[j]=true;           //在时间j销售商品i
                ans+=prod[i].p;         //累计利润
                break;
            }
        }
    }
    return ans;
}

```

```
int main() {  
    int n;  
    while(~scanf("%d",&n)) {  
        if(n==0) {  
            printf("0\n");  
            continue;  
        }  
        int maxd=0;  
        for(int i=0;i<n;i++) {  
            scanf("%d%d",&prod[i].p,&prod[i].d);  
            maxd=max(maxd,prod[i].d);  
        }  
        printf("%d\n",schedule(n,maxd));  
    }  
    return 0;  
}
```

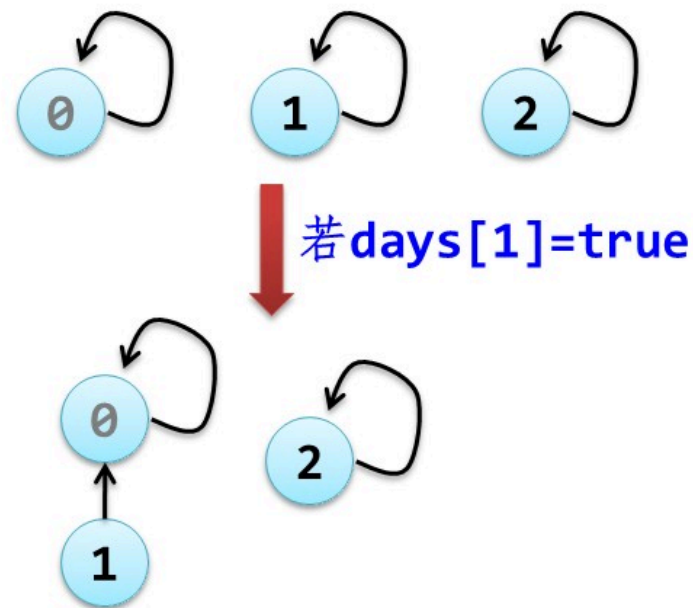
EXE

执行用时为 **297ms**，内存消耗为 **180KB**。

算法改进

- 可以采用并查集提高性能。
- 设置并查集数组为 $\text{parent}[0..\text{maxd}]$ ，其中 $\text{parent}[i]$ 表示截止时间 i 前面最晚的空时间，初始时置 $\text{parent}[i]=i$ ，当在时间 j 销售一个商品时置 $\text{parent}[j]=j-1$ 。

$(\text{pa}, \text{da})=(50, 2)$
 $(\text{pb}, \text{db})=(10, 1)$
 $(\text{pc}, \text{dc})=(20, 2)$
 $(\text{pd}, \text{dd})=(30, 1)$
 $\text{maxd}=2$




```
int parent[MAXN];           //并查集存储结构
int Find(int x) {           //并查集中查找x结点的根结点
    if (x!=parent[x])
        parent[x]=Find(parent[x]); //路径压缩
    return parent[x];
}
```

<code>int schedule(int n,int maxd) {</code>	<code>//贪心算法</code>
<code>for(int i=0;i<=maxd;i++)</code>	<code>//初始化并查集</code>
<code>parent[i]=i;</code>	
<code>sort(prod,prod+n);</code>	<code>//按p递减排序</code>
<code>int ans=0;</code>	<code>//存放答案</code>
<code>for(int i=0;i<n;i++) {</code>	
<code>int di=Find(prod[i].d);</code>	
<code>if(di>0) {</code>	<code>//能够销售</code>
<code>ans+=prod[i].p;</code>	
<code>parent[di]=di-1;</code>	<code>//合并</code>
<code>}</code>	
<code>}</code>	
<code>return ans;</code>	
<code>}</code>	

EXE

执行用时为**63ms**，内存消耗为**212KB**。

解法2

- **贪心策略**：优先考虑晚的截止时间，在所有能够销售的商品中选择利润最大的商品进行销售。
- 用**prod**数组存放全部商品的数据，将**prod**按**截止日期递减排序**。
- **maxd**从最大截止日期**prod[0].d**到**0**循环：将截止日期大于或者等于**maxd**的所有商品的利润值进入到大根堆**pq**中，然后出队最大利润值并累计到**ans**中，最后返回**ans**即可。

```

1. #include<iostream>
2. #include<queue>
3. #include<algorithm>
4. using namespace std;
5. #define MAXN 10005
6.
7. struct Products {                                //商品的类型
8.     int d;                                        //截止时间
9.     int p;                                        //利润
10. bool operator<(const Products&o) {
11.     return d>o.d;                                //按截止时间递减排序
12. }
13. };
14. Products prod[MAXN];                            //存放全部商品

```



```

15. int schedule(int n) {                                //贪心算法
16.     sort(prod,prod+n);                              //按截止时间递减排序
17.     int ans=0;                                       //答案
18.     priority_queue<int> pq;                          //按利润越大越优先出队
19.     int i=0;                                         //i依次遍历prod
20.     for(int day=prod[0].d;day>0;day--) {
21.         for(;i<n && prod[i].d>=day;i++)
22.             pq.push(prod[i].p);                    //将截止时间≥day的商品利
                                                    润进队
23.         if(!pq.empty()) {
24.             int p=pq.top(); pq.pop();                //选择销售利润最大的
                                                    商品
25.             ans+=p;
26.         }
27.     }
28.     return ans;
29. }

```

```
30.int main() {
31.  int n;
32.  while(~scanf("%d",&n)) {
33.      if(n==0) {
34.          printf("0\n");
35.          continue;
36.      }
37.      for(int i=0;i<n;i++)           //输入数据
38.          scanf("%d%d",&prod[i].p,&prod[i].d);
39.      printf("%d\n",schedule(n));    //输出结果
40.  }
41.  return 0;
42. }
```



执行用时为**63ms**，内存消耗为**316KB**。